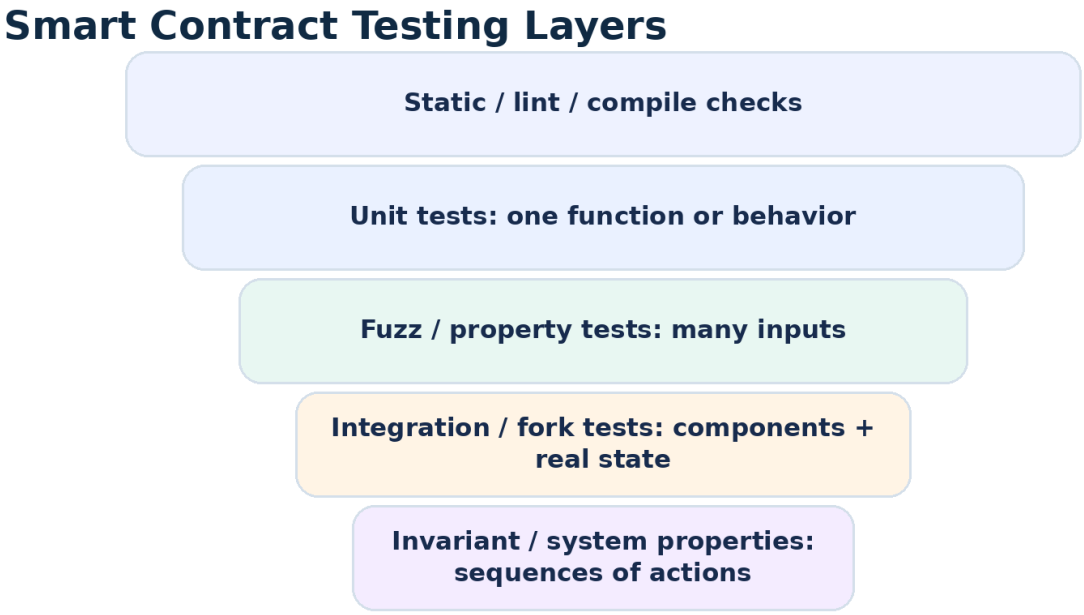
DAY 9

Smart Contract Testing: Unit, Revert, Event, Fuzz & Integration Tests

Foundry / Forge workflow for job-ready Solidity developers

60-MINUTE LECTURE Beginner -> professional testing mindset. The goal is to stop treating deployment as the test and start delivering reproducible evidence that contract behavior matches requirements.



CAREER OUTCOME After this lecture, you should be able to open a client repository, identify the testing framework, add high-value tests, reproduce a failure, and explain what is proven - and what is not.

1. Learning Objectives & Minute-by-Minute Schedule

By the end of Day 9, students should be able to write and run a professional beginner-level Solidity test suite, diagnose common failures, and communicate test evidence to a client.

- Explain why smart contracts require stronger testing discipline than ordinary scripts: deployed code can control irreversible asset movement and persistent on-chain state.
- Use the Arrange-Act-Assert pattern and a deterministic setUp() fixture.
- Write happy-path unit tests, negative/revert tests, event tests and balance/state assertions.
- Use Foundry test actors and cheatcodes such as prank, deal, expectRevert, expectEmit, warp and roll.
- Write fuzz tests where input parameters are automatically generated and constrained to useful domains.
- Explain the purpose of invariant testing, fork testing, coverage and gas snapshots.
- Use verbose traces and targeted test filters to debug failures quickly.
- Translate a client requirement into a test matrix before deployment.

Time	Lecture block	Output
0-5 min	Why testing matters	Testing mental model
5-12 min	Test structure + Foundry	Run first deterministic test
12-20 min	Assertions + actors	State, caller and ETH balance checks
20-28 min	Reverts + events	Negative behavior and logs
28-37 min	Fuzz testing	Properties across many inputs
37-43 min	Time/block + invariants	Stateful edge cases
43-49 min	Integration/fork/mocks	External dependency strategy
49-56 min	Hands-on PaymentVault suite	Client-style test file
56-60 min	Debugging + freelancer checklist	Explain evidence + homework

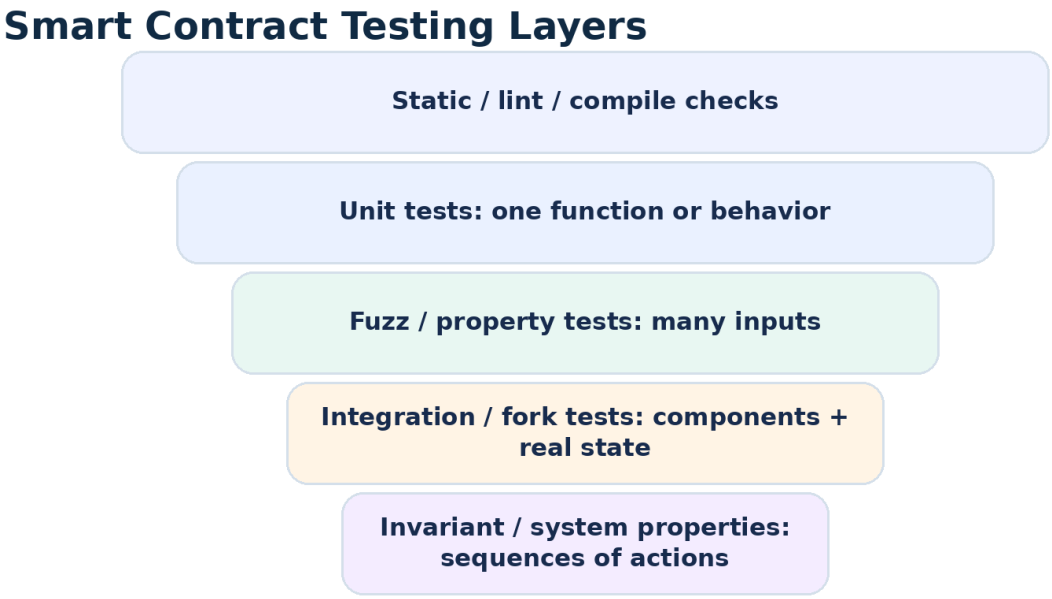
INSTRUCTOR NOTE The PDF is intentionally more detailed than 60 minutes of spoken material. Teach the timed core, use code pages during the lab, and keep the extra reference pages for review/homework.

2. Why Testing Is a Deliverable - Not an Optional Extra | 0-5 min

A smart contract can compile perfectly and still be wrong. Compilation proves syntax/type correctness; tests provide repeatable evidence about behavior under selected conditions.

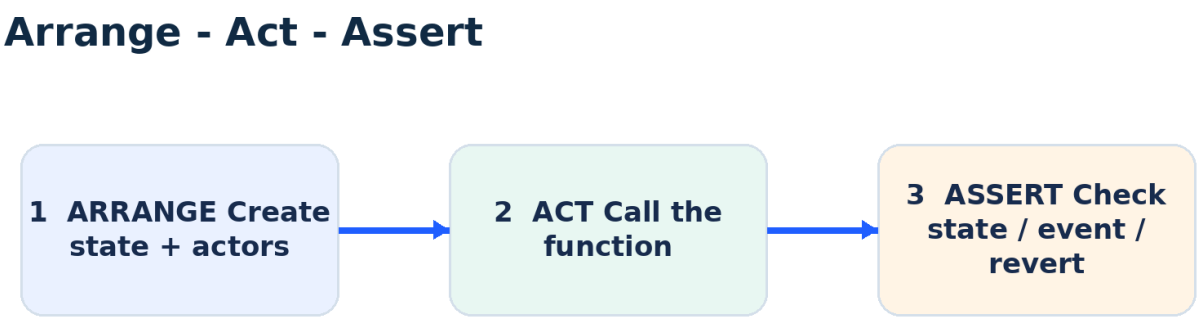
What a good test suite should answer

- Does the expected user action succeed?
- Does unauthorized or invalid behavior fail?
- Does contract state change exactly as required?
- Are emitted events correct for indexers/frontends?
- Are ETH/token balances correct before and after execution?
- What happens at boundaries: zero, maximum, repeated calls, wrong caller, wrong state, wrong time?
- Does the contract still behave correctly when connected to external protocols or a forked network state?



IMPORTANT Passing tests do not prove a contract is secure. They prove only the behaviors and properties actually exercised by the suite. Security review, adversarial analysis and audits remain separate layers.

3. Test Anatomy: Arrange - Act - Assert | 5-8 min



This pattern keeps tests readable for clients and reviewers. Each test should make the starting conditions, action and expected result obvious.

```
function test_DepositTracksUserBalance() public {
    // Arrange
    uint256 amount = 2 ether;
    vm.deal(alice, amount);

    // Act
    vm.prank(alice);
    vault.deposit{value: amount}();

    // Assert
    assertEq(vault.balances(alice), amount);
    assertEq(address(vault).balance, amount);
}
```

Naming tests for fast diagnosis

Pattern	Example	Why it helps
test_<behavior>	test_DepositIncreasesBalance	Readable success case
test_RevertWhen_<condition>	test_RevertWhen_DepositIsZero	Failure intent is explicit
testFuzz_<property>	testFuzz_DepositTracksAnyPositiveAmount	Signals generated inputs
invariant_<property>	invariant_TrackedNeverExceedsEthBalance	Signals a system property

4. Foundry / Forge Test Workflow | 8-12 min

Forge runs Solidity tests from the test/ directory. A common test contract inherits Test from forge-std, deploys the contract under test in setUp(), then uses assertions and cheatcodes to create controlled scenarios.

```
project/
├── foundry.toml
├── src/
│   └── FreelancePaymentVault.sol
└── test/
    └── FreelancePaymentVault.t.sol
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {Test} from "forge-std/Test.sol";
import {FreelancePaymentVault} from "../src/FreelancePaymentVault.sol";

contract FreelancePaymentVaultTest is Test {
    FreelancePaymentVault vault;
    address alice = makeAddr("alice");

    function setUp() public {
        vault = new FreelancePaymentVault();
    }
}
```

Commands worth memorizing

Command	Purpose
forge test	Run the test suite
forge test -vvv	Show richer logs/traces
forge test -vvvv	Maximum verbosity for deep debugging
forge test --match-test test_Name	Run one matching test
forge test --match-contract ContractName	Run one test contract
forge coverage	Generate code coverage information
forge snapshot	Record gas usage snapshots

CURRENT TOOLING NOTE Foundry supports Solidity-native tests, fuzz tests, invariant tests and fork tests. Hardhat 3 also supports Solidity and TypeScript testing; knowing the test concepts matters more than memorizing one framework.

5. Assertions: Prove the Result, Not Just the Transaction

A weak test calls a function and assumes success means correctness. A strong test checks all important postconditions.

Assertion goal	Typical check
Exact value	assertEq(actual, expected)
Boolean condition	assertTrue(condition)
Inequality / ordering	assertGt, assertGe, assertLt, assertLe
Approximate values	assertApproxEqAbs / assertApproxEqRel when appropriate
Contract/user balance	address(x).balance or token.balanceOf(x)
Stored state	public getter or explicit view function

```
function test_DepositUpdatesEveryObservableResult() public {
    uint256 amount = 1.5 ether;
    vm.deal(alice, amount);

    vm.prank(alice);
    vault.deposit{value: amount}();

    assertEq(vault.balances(alice), amount);
    assertEq(vault.totalTracked(), amount);
    assertEq(address(vault).balance, amount);
    assertEq(alice.balance, 0);
}
```

Beginner rule

ASSERT THE REQUIREMENT If the client requirement says “the user balance, total accounting and contract ETH must all update,” write checks for all three. Do not settle for one convenient assertion.

6. Test Actors: msg.sender, ETH Funding & Controlled Callers | 12-16 min

Most smart-contract bugs are caller-dependent. Your test must intentionally control who calls each function and how much ETH/tokens they own.

Tool / idea	Use
makeAddr("alice")	Create a deterministic labeled test address
vm.deal(alice, 10 ether)	Give an address native ETH for the test
vm.prank(alice)	The next external call uses alice as msg.sender
vm.startPrank(alice)	Multiple following calls use alice as msg.sender
vm.stopPrank()	End the persistent prank

```
function test_TwoUsersHaveIndependentBalances() public {
    vm.deal(alice, 3 ether);
    vm.deal(bob, 5 ether);

    vm.prank(alice);
    vault.deposit{value: 3 ether}();

    vm.prank(bob);
    vault.deposit{value: 5 ether}();

    assertEq(vault.balances(alice), 3 ether);
    assertEq(vault.balances(bob), 5 ether);
    assertEq(vault.totalTracked(), 8 ether);
}
```

FREELANCER HABIT Create explicit actor names - client, admin, buyer, seller, attacker, treasury - rather than scattering raw addresses through a test file. Client requirements map naturally to actors.

7. Negative Tests: Reverts Are Part of the Specification | 16-23 min

A secure contract is defined both by what it allows and what it rejects. Negative tests should verify the exact reason or selector whenever practical.

```
function test_RevertWhen_DepositIsZero() public {
    vm.prank(alice);
    vm.expectRevert(FreelancePaymentVault.ZeroAmount.selector);
    vault.deposit{value: 0}();
}

function test_RevertWhen_WithdrawExceedsBalance() public {
    vm.deal(alice, 1 ether);
    vm.prank(alice);
    vault.deposit{value: 1 ether}();

    vm.prank(alice);
    vm.expectRevert(
        abi.encodeWithSelector(
            FreelancePaymentVault.InsufficientBalance.selector,
            1 ether,
            2 ether
        )
    );
    vault.withdraw(2 ether);
}
```

Common negative-test dimensions

- Wrong caller / missing role
- Zero address
- Zero amount
- Amount too large
- Function called twice
- Wrong lifecycle state
- Too early / too late
- Paused state
- Invalid signature / expired deadline
- External call failure

8. Events & Logs: Test What Frontends and Indexers Depend On | 23-28 min

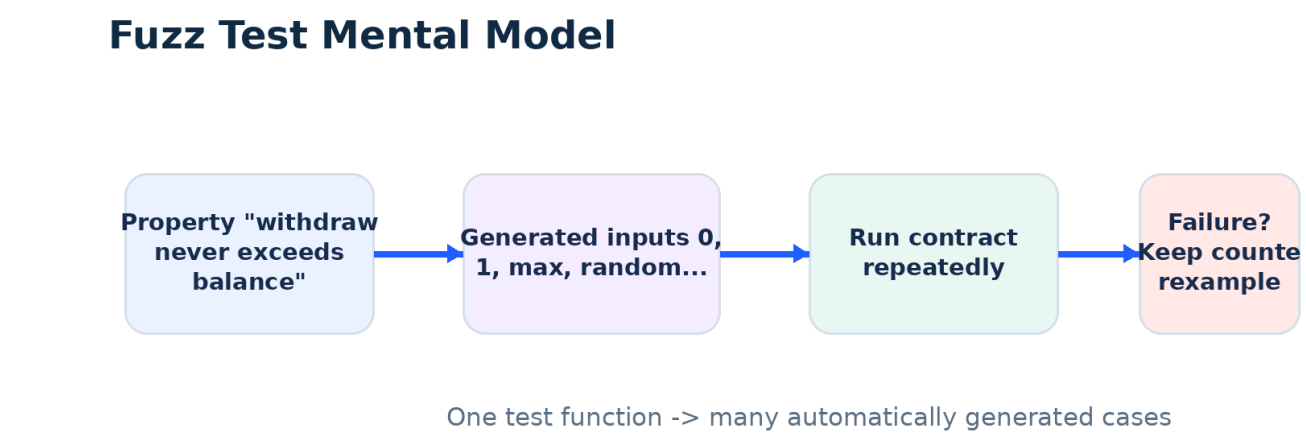
Events are part of many client-facing APIs. A transaction can update state correctly but still break a frontend or indexer if the event fields are wrong or missing.

```
event Deposited(address indexed user, uint256 amount);

function test_EmitsDeposited() public {
    uint256 amount = 2 ether;
    vm.deal(alice, amount);

    vm.expectEmit(true, false, false, true);
    emit FreelancePaymentVault.Deposited(alice, amount);

    vm.prank(alice);
    vault.deposit{value: amount}();
}
```

expectEmit flag	Meaning in this event
checkTopic1 = true	Check indexed user
checkTopic2 / 3 = false	No second or third indexed parameter
checkData = true	Check non-indexed amount data

CLIENT VIEW If a dApp listens for Deposited(user, amount), event tests protect the integration contract between Solidity and the frontend/backend.

9. Fuzz Testing: Test Properties Across Many Inputs | 28-34 min



A Foundry test function with parameters can be fuzzed automatically. Instead of choosing only 1 ether and 2 ether, the tool explores many generated values and reports a counterexample if the property fails.

```
function testFuzz_DepositTracksAnyPositiveAmount(uint96 rawAmount) public {
    uint256 amount = bound(uint256(rawAmount), 1, 100 ether);
    vm.deal(alice, amount);

    vm.prank(alice);
    vault.deposit{value: amount}();

    assertEq(vault.balances(alice), amount);
    assertEq(vault.totalTracked(), amount);
    assertEq(address(vault).balance, amount);
}
```

Unit example vs property

Example-based unit test	Property/fuzz test
Deposit exactly 1 ether	For every allowed positive amount, accounting equals the deposit
Withdraw exactly 0.5 ether	For every withdrawal <= balance, remaining balance = old balance - amount
One chosen boundary	Many generated values, including unexpected combinations

10. Constrain Inputs Without Hiding Bugs | 34-37 min

Random inputs are useful only when the domain represents valid or intentionally invalid behavior. Two common techniques are bounding and assumptions.

Prefer bound when you can map values into a useful range

```
uint256 amount = bound(rawAmount, 1 wei, 100 ether);
```

Use assumptions when values outside the domain should be discarded

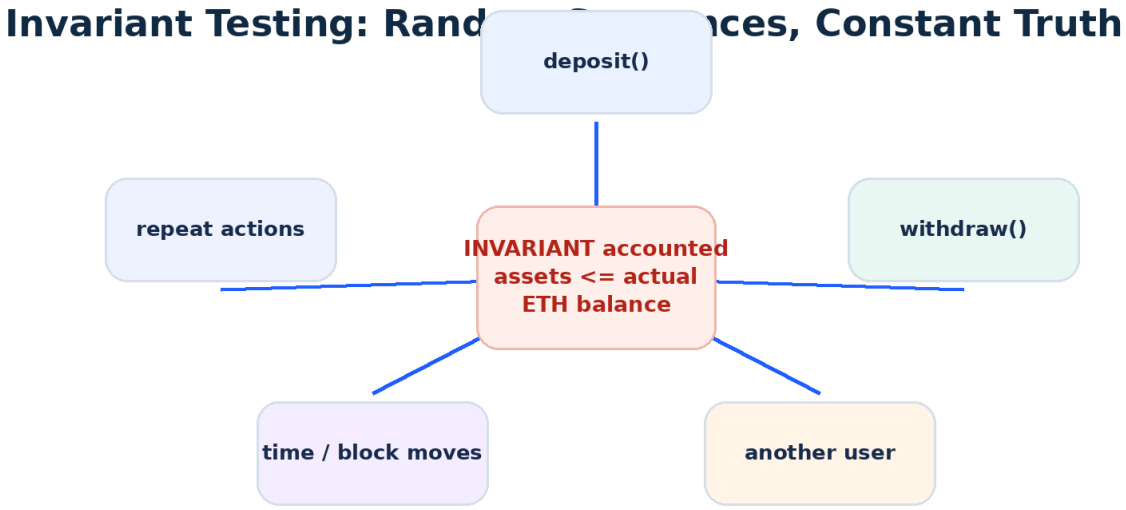
```
vm.assume(user != address(0));
vm.assume(user != address(vault));
```

ANTI-PATTERN Do not add broad assumptions merely to make a failing fuzz test pass. A rejected input may be showing you a real boundary bug.

Good fuzz properties

- Conservation/accounting: tracked balances move by exactly the transferred amount.
- Access control: an unauthorized actor never succeeds.
- Monotonicity: a counter or total only moves in the allowed direction.
- Upper/lower bounds: a value never exceeds the configured cap.
- Round-trip behavior: deposit then valid withdraw returns expected state.

11. Invariant Testing: What Must Always Remain True? | 37-40 min



Invariant testing is stateful. Instead of one call with many inputs, the test framework explores sequences of actions and repeatedly checks a property that should remain true.

```
function invariant_TrackedNeverExceedsEthBalance() public view {
    assertLe(vault.totalTracked(), address(vault).balance);
}
```

For a production protocol, handlers usually constrain the actions the fuzzer can perform. Day 9 focuses on the mental model; we will revisit adversarial security testing later.

WHY "<=" INSTEAD OF "=="? A contract can receive ETH in ways that bypass ordinary deposit accounting (for example, forced ETH from another contract). Tests should encode the property that is actually guaranteed by the implementation.

12. Time, Blocks & Stateful Edge Cases | 40-43 min

Contracts for vesting, auctions, staking, deadlines and governance often depend on block.timestamp or block.number. Waiting in real time is not a testing strategy.

Cheatcode	Purpose
vm.warp(newTimestamp)	Set block.timestamp for the test environment
vm.roll(newBlockNumber)	Set block.number
skip(seconds)	Convenience helper in forge-std for moving time forward
rewind(seconds)	Convenience helper for moving time backward when appropriate

```
function test_WithdrawalUnlocksAfterDeadline() public {
    uint256 unlock = block.timestamp + 7 days;
    Timelock lock = new Timelock(unlock);

    vm.expectRevert(Timelock.TooEarly.selector);
    lock.withdraw();

    vm.warp(unlock);
    lock.withdraw();
}
```

BOUNDARY TEST Test exactly before the deadline, exactly at the deadline, and after the deadline. Off-by-one comparisons (< vs <=) are common smart-contract defects.

13. Integration, Mocks & Fork Testing | 43-49 min

Unit tests isolate your contract. Integration tests verify contracts together. Fork tests execute locally against a snapshot of real chain state - useful when your project integrates DEX routers, lending pools, tokens or oracles.

Fork Testing Architecture

Tests execute locally while reading a chosen block/state from a live network snapshot

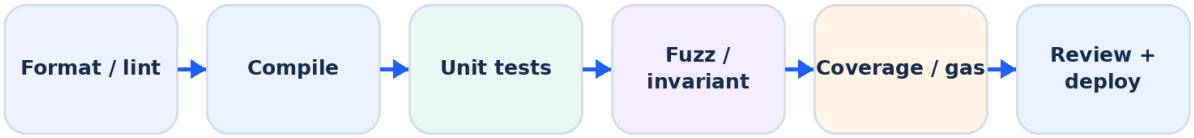
Strategy	Best use	Tradeoff
Mock / stub	Fast deterministic unit tests for an external dependency	May not perfectly match production behavior
Local multi-contract integration	Test your own contracts together	More setup and state interactions
Fork test	Exercise real deployed protocol contracts/state	Depends on RPC/state assumptions; slower/more complex

```
# CLI approach
forge test --fork-url $ETH_RPC_URL

# Tests can also create/select forks programmatically when needed.
```

CLIENT PRACTICE Pin a known block number for reproducible fork tests when the exact external state matters. Document the network, block and RPC assumptions in the repository.

14. Coverage, Gas & the Testing Pipeline

Client-Ready Testing Pipeline



A green test command is evidence - not proof of total correctness.

Coverage

Coverage helps find lines/branches the suite never executes. High coverage can be useful, but 100% coverage is not equivalent to 100% correctness: weak assertions can execute code without proving the right behavior.

Gas snapshots

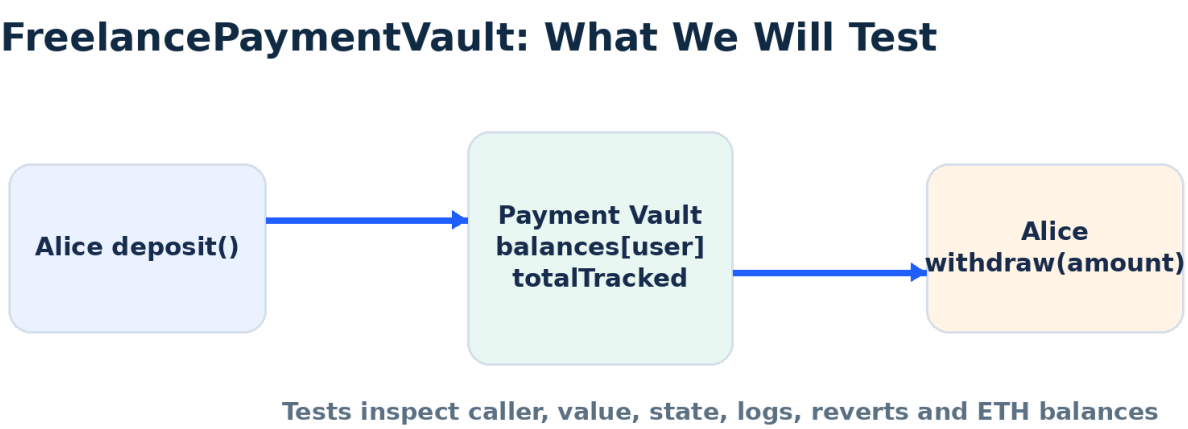
Gas snapshots make cost changes visible during development. Treat them as regression signals, not as a reason to sacrifice readability or security for tiny savings.

```
forge coverage
forge snapshot
forge test -vvvv
```

A practical pre-delivery gate

- Formatting/linting clean
- Compilation succeeds with intended compiler settings
- All deterministic tests pass
- Fuzz/invariant tests pass at configured run counts
- Coverage gaps reviewed
- Gas regressions reviewed
- Deployment configuration and addresses reviewed
- Known limitations documented

15. Practical Contract: FreelancePaymentVault | 49-51 min



The contract intentionally uses a small accounting model so beginners can focus on testing. Each user can deposit ETH and later withdraw up to their tracked balance.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract FreelancePaymentVault {
    error ZeroAmount();
    error InsufficientBalance(uint256 available, uint256 requested);
    error TransferFailed();

    event Deposited(address indexed user, uint256 amount);
    event Withdrawn(address indexed user, uint256 amount);

    mapping(address => uint256) public balances;
    uint256 public totalTracked;

    function deposit() external payable {
        if (msg.value == 0) revert ZeroAmount();
        balances[msg.sender] += msg.value;
        totalTracked += msg.value;
        emit Deposited(msg.sender, msg.value);
    }

    function withdraw(uint256 amount) external {
        if (amount == 0) revert ZeroAmount();
        uint256 available = balances[msg.sender];
        if (amount > available) {
            revert InsufficientBalance(available, amount);
        }

        balances[msg.sender] = available - amount;
        totalTracked -= amount;

        (bool ok,) = payable(msg.sender).call{value: amount}("");
        if (!ok) revert TransferFailed();
        emit Withdrawn(msg.sender, amount);
    }
}
```

16. Practical Test Suite - Setup + Happy Paths | 51-53 min

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {Test} from "forge-std/Test.sol";
import {FreelancePaymentVault} from "../src/FreelancePaymentVault.sol";

contract FreelancePaymentVaultTest is Test {
    FreelancePaymentVault vault;
    address alice;
    address bob;

    function setUp() public {
        vault = new FreelancePaymentVault();
        alice = makeAddr("alice");
        bob = makeAddr("bob");
        vm.deal(alice, 100 ether);
        vm.deal(bob, 100 ether);
    }

    function test_Deposit() public {
        vm.prank(alice);
        vault.deposit{value: 4 ether}();

        assertEq(vault.balances(alice), 4 ether);
        assertEq(vault.totalTracked(), 4 ether);
        assertEq(address(vault).balance, 4 ether);
    }

    function test_Withdraw() public {
        vm.prank(alice);
        vault.deposit{value: 5 ether}();

        uint256 beforeBalance = alice.balance;
        vm.prank(alice);
        vault.withdraw(2 ether);

        assertEq(vault.balances(alice), 3 ether);
        assertEq(vault.totalTracked(), 3 ether);
        assertEq(alice.balance, beforeBalance + 2 ether);
    }
}
```

TEACHING PROMPT Ask the student to name every state variable and ETH balance that changes during withdraw() before reading the assertions.

17. Practical Test Suite - Reverts + Events | 53-54 min

```
function test_RevertWhen_DepositIsZero() public {
    vm.expectRevert(FreelancePaymentVault.ZeroAmount.selector);
    vault.deposit{value: 0}();
}

function test_RevertWhen_WithdrawExceedsBalance() public {
    vm.prank(alice);
    vault.deposit{value: 1 ether}();

    vm.expectRevert(
        abi.encodeWithSelector(
            FreelancePaymentVault.InsufficientBalance.selector,
            1 ether,
            2 ether
        )
    );
    vm.prank(alice);
    vault.withdraw(2 ether);
}

function test_EmitsDepositEvent() public {
    vm.expectEmit(true, false, false, true);
    emit FreelancePaymentVault.Deposited(alice, 3 ether);

    vm.prank(alice);
    vault.deposit{value: 3 ether}();
}
```

Order matters

expectRevert and expectEmit configure expectations for the next relevant call/log. prank controls the caller of the next call. Keep the sequence close together so reviewers can see exactly which action each cheatcode applies to.

18. Practical Test Suite - Fuzz + Accounting Property | 54-56 min

```
function testFuzz_DepositThenWithdraw(
    uint96 rawDeposit,
    uint96 rawWithdraw
) public {
    uint256 depositAmount = bound(uint256(rawDeposit), 1, 50 ether);
    uint256 withdrawAmount = bound(uint256(rawWithdraw), 1, depositAmount);

    vm.deal(alice, depositAmount);
    vm.prank(alice);
    vault.deposit{value: depositAmount}();

    vm.prank(alice);
    vault.withdraw(withdrawAmount);

    assertEq(
        vault.balances(alice),
        depositAmount - withdrawAmount
    );
    assertEq(
        vault.totalTracked(),
        depositAmount - withdrawAmount
    );
    assertEq(
        address(vault).balance,
        depositAmount - withdrawAmount
    );
}
```

PROPERTY After any valid deposit followed by any valid withdrawal, all three accounting views agree on the same remaining amount.

Stretch exercise

Add a handler-based invariant suite with multiple users and random sequences of deposits and withdrawals. Define a property that remains valid even if the vault receives unexpected ETH.

19. Debugging a Failing Test | 56-58 min

When a test fails, reduce the problem. Do not randomly edit contract code until the test turns green.

1. Run only the failing test with --match-test.
2. Increase verbosity (-vvv or -vvvv) and inspect the call trace.
3. Confirm which address is msg.sender and which account owns the assets.
4. Check the exact revert selector/data rather than relying on a vague error description.
5. Compare expected state against actual state before and after the failing call.
6. For fuzz failures, preserve and understand the counterexample input.
7. Check whether the test assumption is wrong before changing production code.
8. Add a regression test that reproduces the bug before applying the fix.

```
forge test --match-test test_Withdraw -vvvv
forge test --match-contract FreelancePaymentVaultTest -vvv
```

Symptom	Likely first check
Expected revert did not occur	Caller, amount, state and expectation ordering
Unexpected revert	Trace + revert data + actor balance
Event mismatch	Indexed topics vs data fields
Fuzz test rejects too many inputs	Overuse of vm.assume; prefer bound
Fork test changed over time	Pin block/network and verify external contract state
Balance assertion off	Remember gas only matters for broadcasted txs; local test accounting may differ from live-wallet expectations

20. Security Edge-Case Test Matrix

Before a client deployment, turn each requirement and threat into a matrix. This catches omissions more reliably than writing tests ad hoc.

Category	Questions to test	Example
Access	Who can call? Who must fail?	Non-owner cannot pause/mint/upgrade
Amounts	Zero, 1, exact balance, > balance, maximum?	Withdraw 0 and balance+1
Addresses	Zero/self/contract/EOA?	Recipient address(0)
State machine	Valid transitions? repeated actions?	Release twice must fail
Time	Before/at/after deadline?	Claim at unlock timestamp
External calls	Revert/return false/reenter?	Mock token or receiver failure
Events	Correct indexed fields/data?	Transfer/Deposit exact values
Accounting	State and token/ETH balances agree?	Tracked assets <= real assets
Precision	Decimals, rounding, shares?	Smallest amount and dust
Admin lifecycle	Role grant/revoke/renounce?	Old admin loses permission

DAY 10 BRIDGE Tomorrow we turn this testing mindset toward attack surfaces: reentrancy, access-control failures, external calls, transaction ordering and other smart-contract security risks.

21. Upwork / LinkedIn Client Scenarios

Scenario A - “The contract is finished. Just add tests.”

Do not estimate only by line count. First identify critical state transitions, privileged functions, asset flows, external integrations, current framework, compiler settings, existing coverage and deployment assumptions.

Scenario B - “One test fails only sometimes.”

Check time/state dependencies, randomness/fuzz seeds, external RPC/fork state, ordering between tests, leaked shared state and assumptions. A deterministic regression test is the deliverable.

Scenario C - “We have 100% coverage, so are we safe?”

Explain that coverage measures executed code, not assertion quality or adversarial completeness. Review boundary conditions, access control, properties/invariants and integration behavior before claiming confidence.

Scenario D - “Can you test our DeFi integration without mainnet funds?”

Use local mocks for isolated behavior and fork tests for realistic interactions with deployed protocols. Pin network/block assumptions and never use production keys in tests.

Client-ready response structure

```
1. Scope: contracts + critical behaviors
2. Framework: Foundry / Hardhat / existing project stack
3. Test plan: unit + negative + fuzz + integration/fork
4. Reproduction: exact command and environment
5. Evidence: passing tests, coverage/gas notes, known gaps
6. Delivery: code + README + CI command + limitations
```

22. 60-Minute Hands-On Lab Checklist

Use this as the live class exercise or repeat it independently after the lecture.

1. Create a Foundry project and place FreelancePaymentVault.sol under src/.
2. Create FreelancePaymentVault.t.sol under test/ and import forge-std/Test.sol.
3. Deploy the vault in setUp() and create alice/bob actors with makeAddr.
4. Write a successful deposit test with storage and contract-balance assertions.
5. Write a successful withdraw test that checks the user receives ETH and accounting decreases.
6. Write zero-deposit and insufficient-balance revert tests.
7. Write an event test for Deposited.
8. Write a fuzz test for deposit -> withdraw with bounded values.
9. Run the full suite, then intentionally break one assertion and inspect -vvvv output.
10. Create a short README section called “Testing” with the exact commands a client should run.

Definition of done

- A fresh clone can install dependencies and run the documented test command.
- No private keys or RPC secrets are committed.
- Tests have descriptive names and clear actors.
- At least one negative test exists for every important require/revert branch.
- Known untested areas are documented instead of hidden.

23. Interview Questions + Mini Quiz

Interview questions - answer verbally

Question	Strong answer should mention
Unit vs integration test?	Isolation of one behavior vs multiple components/external dependencies
Why test reverts?	Rejected behavior is part of the contract specification/security boundary
What is fuzz testing?	Generated inputs used to test a general property and expose counterexamples
What is an invariant?	A property that must remain true across sequences of state-changing actions
Why fork test?	Exercise real deployed contracts/state locally without spending live funds
Is 100% coverage enough?	No - execution coverage does not guarantee strong assertions or security
What does vm.prank do?	Controls msg.sender for the next call in a Foundry test
How do you debug a fuzz failure?	Inspect/preserve counterexample, reduce case, fix property/implementation, add regression test

10-question quiz

1. What are the three stages in Arrange-Act-Assert?
2. Which Foundry function prepares shared state before each test?
3. Why is checking only “the call did not revert” usually insufficient?
4. What does expectRevert verify?
5. What is the difference between prank and deal?
6. Why might bound() be preferable to broad vm.assume() conditions?
7. What kind of test explores sequences of actions?
8. Why pin a block for a fork test?
9. Does 100% line coverage prove a contract is secure?
10. What should you do before fixing a newly discovered bug?

24. Quiz Answers, Homework & Portfolio Deliverable

Quiz answers

1. Arrange starting state, Act by calling the behavior, Assert the expected result.
2. setUp().
3. Because correct state, balances, events and other postconditions can still be wrong.
4. That the next relevant call reverts with the expected selector/data/reason.
5. prank changes the caller; deal changes an address native ETH balance for testing.
6. bound maps values into a useful range without rejecting large portions of generated inputs.
7. Invariant/stateful testing.
8. To make external state reproducible as the live chain continues changing.
9. No. Coverage measures execution, not correctness, assertion strength or attack completeness.
10. First add/reproduce it with a failing regression test.

Homework

- Implement the complete FreelancePaymentVault contract and test suite without copying the PDF line-for-line.
- Add tests for two independent users depositing and withdrawing in different orders.
- Add a fuzz test that performs two deposits then one withdrawal and proves the final accounting.
- Add a malicious receiver contract that rejects ETH and test TransferFailed().
- Run forge coverage and note which branches remain uncovered.
- Run forge snapshot, change one function implementation, and compare gas snapshots.
- Write a 6-10 line README “Testing” section suitable for an Upwork delivery.

Portfolio artifact

GITHUB PROJECT Repository title: solidity-payment-vault-testing. Include src/, test/, foundry.toml, README, test commands, coverage note, gas snapshot note and a short “What I tested” matrix. Never publish real keys.

25. Day 9 Cheat Sheet + Primary References

Goal	Pattern	Goal	Pattern
Run all tests	forge test	Debug deeply	forge test -vvvv
One test	--match-test <name>	Shared fixture	setUp()
Actor	makeAddr + vm.prank	Fund actor	vm.deal
Expected revert	vm.expectRevert	Expected event	vm.expectEmit
Move time	vm.warp / skip	Move block	vm.roll
Fuzz input	test parameter	Constrain fuzz	bound / vm.assume
Invariant	invariant_<property>()	Fork state	--fork-url / cheatcodes
Coverage	forge coverage	Gas regression	forge snapshot

Testing mindset

```
Requirement
   ↓
Observable behavior
   ↓
Happy path + rejection path
   ↓
Boundary / fuzz / state sequence
   ↓
Assertions on state + balances + events
   ↓
Reproducible command
   ↓
Client evidence + known limitations
```

Primary references

Reference	Reference
Foundry - Testing https://getfoundry.sh/forge/testing	Foundry - forge test reference https://getfoundry.sh/reference/forge/test
Foundry - Cheatcodes https://getfoundry.sh/reference/cheatcodes/overview	Foundry - Invariant Testing https://getfoundry.sh/guides/invariant-testing
Foundry - Fork Testing https://getfoundry.sh/guides/fork-testing	Foundry - Best Practices https://getfoundry.sh/best-practices
Hardhat 3 - Testing Overview https://hardhat.org/docs/guides/testing	Solidity - Security Considerations https://docs.soliditylang.org/en/latest/security-considerations.html
Solidity - SMTChecker / Formal Verification https://docs.soliditylang.org/en/latest/smtchecker.html	

NEXT: DAY 10 Smart Contract Security I - reentrancy, access control, CEI, external calls, front-running/transaction-ordering concepts, oracle risk introduction, signature replay and secure review workflow.